Resolve.

PAYMENT AFTERCARE FOR PAYPAL MERCHANTS

Technical documentation

Resolve connects a customer's issue, the merchant's commitment and the payment evidence needed to determine whether that commitment was fulfilled.

EDITION	ISSUED	AUDIENCE
Submission	12 November 2026	Reviewers / developers

This guide describes the implemented repository and its recorded validation. PayPal transactions discussed here use Sandbox test funds. A model recommendation does not authorize a refund.

Project overview

The problem

A payment can complete while the customer's problem remains unresolved. The support conversation may contain a refund promise, the store may still show the original payment, and PayPal may hold a different refund state. Reconstructing those records takes time and can lead to repeated requests or duplicate action.

Resolve gives a merchant one case in which to record the issue, identify the agreed remedy, connect the relevant records and check the resulting outcome. The buyer gets a private view of progress and can continue the conversation without receiving merchant credentials.

What is implemented

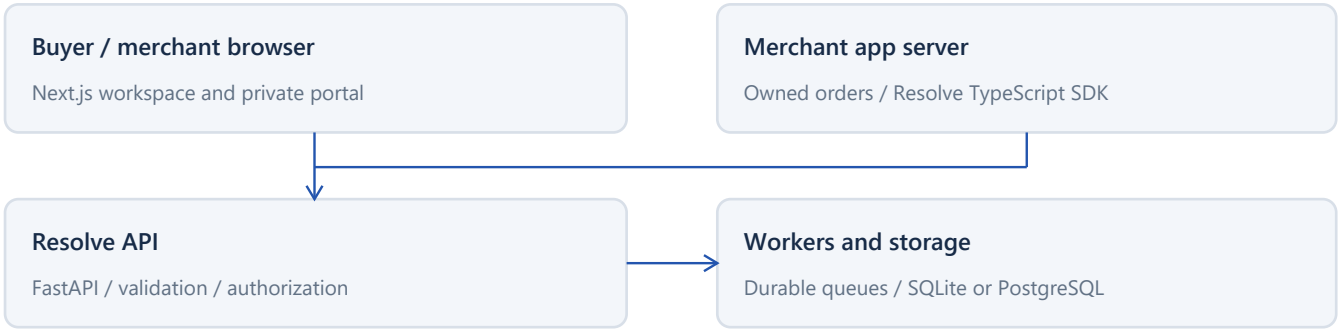
- A merchant workspace for cases, orders, commitments, evidence and operational review.
- Private buyer links for issue intake, messages, attachments, offers and return requests.
- PayPal Sandbox evidence reads and separately authorized refund execution.
- A server-side TypeScript SDK for order registration and embedded aftercare.
- Optional AI extraction and evidence investigation, with bounded outputs and merchant review.
- SQLite or PostgreSQL storage, durable worker queues, local tests and Postman simulations.

Reading guide

Section	Page
System architecture	3
Data model and reconciliation	4
Merchant and buyer workflow	5
Local installation	6
Configuration and access	7
API reference	8
SDK and storefront integration	9
AI investigation	10
Postman contracts and simulations	11
Validation record	12
Operations and troubleshooting	13
Submission contents and references	14

System architecture

The frontend, API and workers are separate processes. The merchant application keeps its own order records and calls Resolve from its server. PayPal credentials remain on trusted servers.



Trusted adapters: PayPal REST / optional evidence providers / bounded model inference

Components and responsibilities

Component	Responsibility
Next.js / React / TypeScript	Merchant workspace, buyer portal and a server-side API gateway. Local CSS provides layout and design tokens.
Python / FastAPI / Pydantic	Authentication, request validation, case workflows, reconciliation and guarded execution.
SQLite / PostgreSQL	Durable cases, observations, approvals, executions, audit records and worker queues. SQLite is the local default.
TypeScript SDK 0.7	Order sync, evidence access and explicitly granted support actions from a merchant server.
Python workers	Investigation, operations, follow-up and refund recovery run independently of browser requests.
Provider adapters	PayPal REST; optional Shopify, EasyPost, Zendesk, email and model integrations.

Trust boundaries

The browser never supplies the merchant's ownership identity. The API derives it from authenticated access. Store servers must also verify that their signed-in customer owns an order before invoking support methods. A provider reference is a link to a record, not proof that the associated action completed.

Source: frontend/package.json; backend/pyproject.toml; backend/app; sdk/typescript/package.json.

Data model and reconciliation

Main records

Record	Meaning
Merchant and access grant	Tenant ownership, individual roles, sessions and restricted app permissions.
Commerce order	An app-owned order snapshot, revision, products and linked payment references.
Resolution case	The customer issue and the common context for messages, evidence and remedies.
Promise / commitment	The agreed action, amount where applicable, deadline, conditions, source and revision.
Evidence observation	A merchant-reported event or provider-derived fact, retaining its source and observation time.
Refund approval	A persisted authorization for an exact action against a particular eligible state.
Refund execution	The durable submission identity, provider outcome, recovery state and execution events.
Investigation	A revision-bound evidence review, findings, missing information and review outcome.

Financial rules

Amounts use explicit precision. SDK callers supply decimal strings such as `"8.00"`; they must not substitute binary floating-point estimates. Currency, seller and payment references must agree with the supported provider workflow.

A capture may be linked to more than one case. Remaining refundable capacity is therefore calculated across that capture's relevant history. Case-local totals alone cannot authorize another refund. Repeated provider IDs and repeated event identities must not create duplicate financial records.

Pending refunds remain distinguishable from completed refunds. Failed or denied refunds do not fulfill the promise. A timeout after submission may mean the provider outcome is unknown; recovery must reconcile the saved identity before deciding whether another submission is permitted.

Missing or stale verification is not a confirmed zero. Public customer responses can return null amounts with verification status and check time. Acceptance, required return conditions, dispute holds, current evidence and exact approval remain separate gates.

Database changes

Numbered migrations are immutable once present. Follow-up schema changes belong in a new migration and should be verified on disposable storage. The current application selects PostgreSQL with `RESOLVE_DATABASE_URL`; otherwise it uses its configured SQLite file.

Source: backend/README.md; backend/app/models.py; backend/app/storage.py; backend/app/postgres.py.

Merchant and buyer workflow



From an order to a resolution

1. Register the order. The merchant app publishes its order snapshot and payment references through the SDK. Registering an ordinary order does not require opening a support case.

2. Record the issue. The buyer uses the store's support entry point or a private intake link. The merchant can also create a case and link the correct order.

3. Establish the commitment. Record the proposed or agreed remedy and its source. Return requirements, acceptance and deadlines belong to the commitment, rather than a free-form financial instruction.

4. Check the evidence. Refresh supported provider records and compare amount, currency, seller, payment identity and refund history. Preserve mismatches and unavailable checks for review.

5. Approve, then execute. An eligible refund receives an exact persisted approval. Submission is a separate action, rechecked against the current state and remaining balance.

6. Verify the outcome. Reconcile the provider result, update the timeline and show the buyer the supported status. Provider completion is distinct from confirmation that funds reached a bank account.

Who can do what

Participant	Primary actions
Buyer	Report an issue, supply evidence, exchange messages and view private progress.
Merchant staff	Investigate and manage cases according to assigned role; financial actions require the applicable permission and gates.
Merchant app server	Sync owned order data and invoke only the methods granted to its app key.
Investigation worker	Gather permitted evidence and propose or abstain. It cannot approve or send a refund.

Demonstrated journey

The submission film follows a USD 8.00 Sandbox purchase through buyer approval, capture, support, merchant refund approval and full refund. The recorded final state is USD 8.00 refunded and USD 0.00 remaining. This is a scoped test-environment demonstration, not a production payment claim.

Local installation

Prerequisites

Use Python 3.12 or later, uv, npm and a supported Node.js runtime. Node.js 24 is a practical common choice for the frontend, SDK and checkout example. The frontend declares Node >=20.9; the SDK declares >=22.12.0. Dependencies and build outputs are excluded from the source ZIP.

1. Start the API

Open a terminal in the extracted `Resolve` folder:

```
cd backend
uv sync
Copy-Item .env.example .env
uv run uvicorn app.main:app --reload
```

Copy the example only for a new installation; preserve an existing private configuration. SQLite works without provider accounts. Interactive API documentation opens at `http://127.0.0.1:8000/docs`. Liveness is `/health`; database readiness is `/ready`.

2. Create merchant access

In another terminal, from the backend directory:

```
uv run python -m app.manage create-merchant --name "Review store"
```

The command prints a token once. Save it privately and use it to connect the workspace or authorize API requests. The database retains its hash. Individual account registration is also implemented at `/account` when enabled.

3. Start the frontend

In a separate terminal, from the extracted root:

```
cd frontend
npm ci
Copy-Item .env.example .env.local
npm run dev
```

Open `http://127.0.0.1:3000`. The example points the server gateway to `http://127.0.0.1:8000`. Sign in to the running API; missing provider configuration appears as unavailable. `/demo` redirects to sign-in.

For a compiled local preview, run `npm run build:preview`, then `npm run preview` from frontend instead of the development server. The build exports the public guides from included frontend content. Rebuild after source or configuration changes.

Configuration and access

Configuration locations

Location	Purpose
backend/.env	Ordinary backend database, PayPal, identity, email and evidence-connector settings. Begin with .env.example.
frontend/.env.local	Server-side API origin and matching merchant/customer origins. Begin with .env.example.
backend/.env.ai.ps1	Optional private PowerShell process settings copied from ai-env.example.ps1. Load explicitly before starting AI processes.
Merchant server environment	Resolve API URL, app source and the correct scoped SDK key. Never publish these in browser code.

Relevant switches

Setting	Meaning / default
RESOLVE_DATABASE_URL	Select PostgreSQL. Omit for the configured local SQLite file.
RESOLVE_PAYPAL_SANDBOX_ENABLED	Enable configured Sandbox evidence reads; false in the example.
RESOLVE_PAYPAL_SANDBOX_REFUNDS_ENABLED	Separate opt-in for approved Sandbox refund writes; false.
RESOLVE_PAYPAL_SANDBOX_PARTIAL_REFUNDS_ENABLED	Additional gate for partial/repeated refunds with current history; false.
RESOLVE_PLATFORM_ENCRYPTION_KEY	Persistent Fernet key for stored provider connections; share securely with the appropriate workers.
RESOLVE_AI_ENABLED	Optional investigation inference; false. A provider key alone does not enable it.

Access model

Full merchant API calls use `Authorization: Bearer <token>`. Browser access uses an opaque HttpOnly session. The system also implements owner, manager, finance, agent and viewer roles. A commerce key is not a full merchant key; support methods need an explicit source-bound grant.

Private buyer links expire and can be revoked or replaced. Another merchant's case is returned as not found. Keep encryption keys with protected backups: changing a key on restart prevents decryption of existing saved connections.

AI settings read process variables only. The investigation worker does not automatically load the API's `.env` file; inject the same storage configuration before starting it. Do not assume that two processes are using the same database.

API reference

The generated OpenAPI contract is the authoritative source for complete schemas, required fields and error responses. The routes below are a reading map, not a replacement contract.

Core route groups

Method and route	Purpose
GET /health; GET /ready	Process liveness and read-only storage readiness.
GET /auth/me	Read the authenticated merchant context.
POST /resolution-cases/upsert	Create or reuse an externally referenced case under the merchant.
POST /resolution-cases/{case_id}/evidence/records	Link a provider or internal record.
POST /resolution-cases/{case_id}/evidence/refresh	Refresh permitted provider evidence for the case.
GET /resolution-cases/{case_id}/evidence-bundle	Read linked records, findings and evidence context.
GET /cases/{case_id}/reconciliation	Read the refund case reconciliation result.
POST /cases/{case_id}/refund-approvals	Create a persisted exact refund approval.
POST /cases/{case_id}/refund-executions	Submit an eligible approved refund through the guarded execution flow.
POST /commerce/orders/sync	Sync a merchant application's order snapshot.
POST /aftercare/issues	Open an owned order's issue with an appropriate support grant.
POST /aftercare/status; POST /aftercare/portal	Read safe order resolution or issue a private support link.

Contract maintenance

The source archive includes `postman/contracts/resolve.openapi.json`. After changing API models or routes, regenerate it from the backend directory:

```
uv run python scripts/export_openapi.py
```

Then refresh the website downloads with `npm --prefix frontend run docs:export` from the root. No regeneration was needed for this documentation change.

Client behavior

Use stable external references and event IDs. Identical SDK upserts, links and events reuse existing records; changed data under a reused identity can return 409. Treat authentication failures, unavailable evidence, stale state and unknown execution outcomes as distinct results. Never interpret an HTTP success from a record-creation endpoint as proof of a PayPal refund.

SDK and storefront integration

Install the local preview

The SDK is a repository package, not a published npm dependency. From the extracted root:

```
cd sdk/typescript
npm ci
npm pack
```

Install the resulting `resolve-paypal-sdk-0.7.0.tgz` in the merchant application's backend. The package builds from the included OpenAPI contract. Its Node client has no runtime dependencies; the optional PostgreSQL example uses an app-supplied reader pool.

Register an order

Generate a commerce integration key in the workspace's SDK settings. Store the copied configuration on the merchant server, including `RESOLVE_API_URL`, `RESOLVE_API_KEY` and a stable `RESOLVE_APP_SOURCE`.

```
import { createResolve } from "@resolve-paypal/sdk";

const resolve = createResolve();
await resolve.syncOrder({
  id: "ORDER-123",
  status: "paid"
});
```

This registers the first revision. Supply the next version for a later update, retaining the same order ID and app source. Include the supported payment and line-item fields when provider comparison is required.

Embed aftercare

`openIssue` records a customer issue; `getOrderResolution` returns safe progress and the next action; `createSupportLink` opens the existing messages, offers and returns portal. Issue a source-bound support key explicitly. An order-only commerce key does not gain these permissions automatically.

The merchant server must authenticate its customer and check order ownership before calling these methods. Keep SDK tokens out of browser bundles and avoid treating a caller-supplied order ID as proof of ownership. Decimal amounts belong in string form where the method accepts them.

Included checkout example

`examples/checkout-store` contains Field Brew Supply, a six-product Sandbox shop. The server quotes catalog prices, creates an order, captures only after buyer approval and syncs the resulting evidence. Its README documents the isolated launcher. Previewing the catalog needs no payment; a new Sandbox purchase still requires deliberate buyer approval.

AI investigation

Purpose and limits

The worker helps a merchant understand the evidence. It can identify a likely commitment, collect permitted linked facts, explain what is missing and propose a next step. It can also abstain. Reviewing its output is separate from approving or submitting a refund.

Processing sequence

1. A case change queues an investigation for the relevant evidence revision.

2. The worker loads the case's permitted evidence and checks ownership, freshness and revision.

3. Bounded model triage produces structured findings or requests for permitted evidence.

4. Backend validators check references, scope, contradictions and proposed actions.

5. Findings or a blocked result are saved for merchant review with the supporting source references.

Freshness alone is insufficient for reuse. A selected cached provider fact must belong to the case and resource, remain valid for the current revision and satisfy the tighter freshness checks. A recent failed refresh cannot make old evidence current.

Configuration and worker

Groq and OpenRouter adapters are implemented. AI is disabled in the example configuration. Direct Groq is blocked by the default `zdr_required` data policy; permitting it requires an explicit compatible policy selection as well as keys and feature flags. This guide does not change those defaults.

From backend, after configuring the private process environment and the same database as the API:

```
. .\.env.ai.ps1
uv run python -m app.investigation_worker --loop
```

Load the intended configuration separately in each terminal. Optional provider refresh requires its own permitted Sandbox settings. Filling a configuration file does not start the worker.

Recorded evidence

The retained Groq-backed investigation of the filmed completed Sandbox case produced three linked provider findings, a confirmed-full-refund/no-action result and zero financial writes. This validates that bounded run. OpenRouter escalation was not verified in that account-backed run.

The extraction evaluation also records zero successful attacks across 50 synthetic injection cases in one run. The chats and labels were AI-authored and have not been independently human-verified. This is a measured dataset result, not a general security guarantee or repeated-run claim.

Postman contracts and simulations

All 28 nonignored Postman files from the source package are retained. The application contract and upstream provider subsets are separate so a caller can distinguish Resolve's API from the HTTP dependencies it reads.

Directory guide

Path under postman/	Contents
contracts/resolve.openapi.json	Generated FastAPI application contract.
contracts/paypal-capture-read.openapi.yaml	Supported PayPal OAuth, capture, refund, history and verification subset.
contracts/resolution-evidence.openapi.yaml	Additional evidence-provider contract subset.
mocks/	PayPal, Shopify, EasyPost, Google, Resend and Zendesk stand-in handlers/configuration.
simulations/	PayPal-read, resolution-workspace, customer-Google, hosted-follow-up and merchant-operations scenarios with context records.
README.md	Full commands, fixture boundaries and interpretation of results.

Run the refund HTTP checks

Install the Postman CLI with simulation support and sync backend dependencies first. The repository records CLI 1.82.0 as the version tested; this is not a claim about the latest available release. From the extracted root:

```
$env:RESOLVE_RUN_PAYPAL_SIMULATION = '1'
$sim = 'postman/simulations/paypal-read/paypal-read.sim.yaml'
$tests = 'uv --directory backend run python -m pytest '
$tests += 'tests/test_paypal_history_simulation.py '
$tests += 'tests/test_paypal_simulation.py '
$tests += 'tests/test_refund_execution_simulation.py -q'
try {
  postman simulation run $sim --proxy --lift $tests --output ndjson
} finally {
  Remove-Item Env:RESOLVE_RUN_PAYPAL_SIMULATION
}
```

The child process uses a temporary local proxy and CA. Tests retain the supported Sandbox URL and TLS verification; normal clients do not adopt environment proxies. Confirm that expected requests reach the stand-in and that zero requests pass through to the real provider.

What a passing simulation establishes

The tests exercise actual application routes, HTTPX behavior and disposable persistence against controlled responses, including errors and unknown outcomes. They do not establish real account permissions, actual email delivery, a received PayPal webhook or hosted production availability. Context files retain those boundaries beside each scenario.

Validation record

This section summarizes retained development evidence. These application tests and account-backed workflows were not rerun to produce this PDF. Documentation verification is limited to content review, rendering and archive integrity.

Evidence by environment

Scope	Recorded result	Interpretation
Isolated backend regression, CHG-060	1,008 passed; 63 opt-in skipped	Local regression result for that recorded code state. Skipped paths are not verified by this total.
PayPal HTTP simulation	Errors, history, duplicate delivery, pending/unknown recovery and guarded execution fixtures	Real caller behavior against local stand-ins; no account-backed permission inferred.
Official PayPal Sandbox	Authorized USD 1.00 workflow and separately filmed USD 8.00 workflow completed	Scoped test purchases, capture and full refunds. No live-money outcome claimed.
Groq investigation, CHG-060	Finished, three verified findings, no action, zero financial writes	One account-backed bounded investigation of the retained Sandbox case.
Synthetic extraction injection set	0 successful attacks / 50 cases in one recorded run	Synthetic, AI-authored labels; no general assurance or independent human validation.
Disposable PostgreSQL audit	12 passed in the recorded integration audit	Local storage/migration/concurrency coverage; hosted storage was not accepted by this run.

Reproduce local checks

From backend, the isolation helper clears inherited provider settings and creates disposable storage:

```
uv run python scripts/isolated.py -m pytest -q
```

From the extracted root, run the included frontend and SDK checks after installing their dependencies:

```
npm --prefix frontend run check
npm --prefix frontend run build
npm --prefix sdk/typescript test
```

Browser suites use Playwright. Follow frontend/README.md for the disposable fixtures and Chromium installation. Provider simulation and account-backed suites require their documented explicit opt-ins; a default suite result does not cover them.

Still to verify

Hosted backend and worker acceptance, actual incoming PayPal webhook delivery, hosted PostgreSQL backup/restore, and relevant external account consent/delivery remain separate work. Optional Shopify, EasyPost, Zendesk, email and Google integrations must be accepted with their own authorized accounts.

Operations and troubleshooting

Process responsibilities

Process	Start from backend/	Role
API	uv run uvicorn app.main:app	Serves authenticated workflows and health/readiness checks.
Investigation	uv run python -m app.investigation_worker --loop	Processes queued case evidence reviews.
Operations	uv run python -m app.operations_worker --loop	Processes operational imports, alerts and projections.
Follow-up	uv run python -m app.followup_worker --loop	Handles configured refresh/notification work; does not submit a refund.
Recovery	uv run python -m app.worker --loop	Reconciles/retries eligible saved refund executions within their authorization and recovery rules.

Use the same intended storage and the required encryption configuration across API and workers. Enable external-service settings deliberately. Development `--reload` is convenient locally; it is not a production process manager.

Common problems

Symptom	First checks
Workspace cannot connect	Confirm the API origin, /ready response, running port and merchant access. Frontend provider errors do not trigger a sample-data fallback.
Refund is unavailable	Check provider freshness, exact approval, acceptance/conditions, capture-wide balance, pending executions, dispute holds and opt-in flags.
A submitted refund times out	Inspect the saved execution and reconcile the same provider identity. Do not create a new request merely because the response was lost.
Investigation stays queued	Start the worker and confirm it points at the API's database. Review privacy policy, inference flags and the recorded blocked reason.
Private buyer link fails	Check expiry, replacement/revocation and the configured customer origin. Use the issued link rather than constructing a query string.
Saved connections cannot decrypt	Check that the original persistent encryption key is available to the process. Avoid replacing it with a new key.

Hosted acceptance

`render.yaml` supplies a deployment blueprint, including the web/API services, workers and PostgreSQL wiring. It is a deployment starting point. Before claiming an operational release, verify origins and TLS, private credentials, tenant isolation, database restore, worker recovery and actual Sandbox callback delivery in the deployed environment.

Submission contents and references

This documentation library contains PDFs only. Application source is a separate delivery. The earlier source-package appendix is replaced here so the reading guide matches the current bundle.

Related volumes

Volume	Use it for
02 Implementation guide	Service/module responsibilities, reconciliation, guarded execution, AI and SDK integration.
03 Database design	Core relational map, every SQLite table/column, constraints, indexes and migration catalog.
04 API reference	All 178 current operations and 251 component schemas from the generated contract.
05 Installation and deployment	Local setup, example settings, identity configuration and hosted readiness.
06 Illustrated user manual	Merchant and buyer procedures, screenshots, practical tutorials and SDK appendix.
07 Testing and security	Historical local, simulated and account-backed results with their limits.
08 Operations and integrations	Staff operations, customer support, provider configuration and Postman tooling.
09 Engineering log	Complete development history, decisions, corrections and actual validation records.

Source references

The guide is grounded in README.md, component READMEs, backend/app, immutable migrations, frontend content, the first-party SDK and postman/contracts/resolve.openapi.json in the project repository. The accompanying books render the relevant maintained documentation, including the full engineering log.

Edition and review sequence

Pages 1-13 retain the 12 November 2026 technical-guide edition. This final page is adapted for the documentation-only library. The original source-submission PDF remains preserved separately. Begin with the overview and illustrated manual, then use the detailed implementation, database and API books. Follow installation instructions against the separately delivered source.

Recorded tests remain historical. The current API operation count differs from the older audit ledger. New database introspection uses disposable SQLite; it is not acceptance of hosted storage. PayPal demonstrations use Sandbox funds, and an AI recommendation never authorizes a financial action.